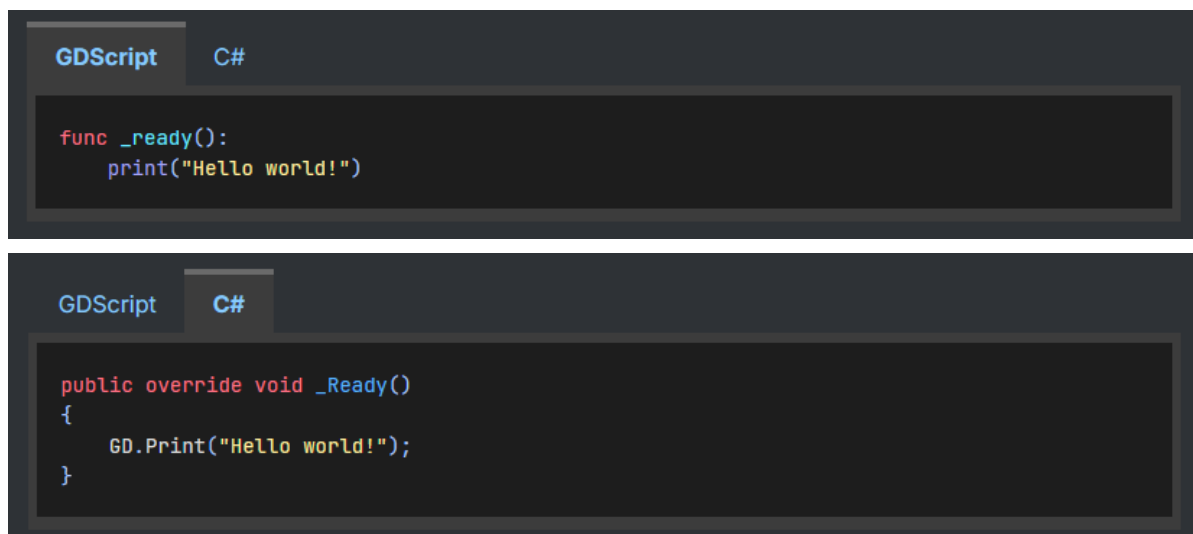


Introdução



Boas-vindas à documentação oficial do **Godot Engine**, a game engine 2D e 3D livre e de código aberto voltada para a comunidade! Por trás disso tudo, você encontrará uma ferramenta poderosa e de fácil utilização que você pode usar para desenvolver qualquer tipo de jogo, para qualquer plataforma e sem nenhuma restrição de uso.

O objetivo dessa página é fornecer uma ampla apresentação da engine e do conteúdo desta documentação, para que você saiba por onde começar se for um iniciante, ou onde procurar se precisar de informações sobre uma funcionalidade específica.

Antes de começar

A página [Tutoriais e recursos](#) lista tutoriais em vídeo fornecidos pela comunidade. Se você prefere vídeo em vez de texto, vale a pena dar uma olhada. Do contrário, os [Primeiros passos](#) são um ótimo ponto de partida.

Caso tenha dificuldades com um dos tutoriais ou com seu projeto, você pode encontrar ajuda em um dos [Canais da comunidade](#), especialmente no [Discord](#) do Godot e no [Fórum](#).

Sobre o Godot Engine

Uma game engine é uma ferramenta complexa e difícil de apresentar em poucas palavras. Aqui está uma sinopse simples, que você pode utilizar sempre que precisar de uma definição rápida do Godot Engine:

O Godot Engine é uma game engine multiplataforma repleta de recursos para criar jogos 2D e 3D a partir de uma interface unificada. Ele fornece um conjunto abrangente de ferramentas comuns para que os usuários possam

simplesmente criar seus jogos sem ter que reinventar a roda. Os jogos podem ser exportados num clique para várias plataformas, incluindo as principais plataformas de desktop (Linux, macOS, Windows), plataformas móveis (Android, iOS), as baseadas na Web e também consoles.

O Godot é completamente livre e de código aberto sob a [permissiva licença MIT](#). Sem compromisso, sem royalties, sem nada. Os jogos dos usuários são deles até a última linha do código da engine. O desenvolvimento do Godot é totalmente independente e voltado para a comunidade, capacitando os usuários a ajudar a moldar sua engine para atender às suas expectativas. Ele é mantido pela [Godot Foundation](#) sem fins lucrativos.

Organização da documentação

Esta documentação é organizada em várias seções:

- **Sobre** contém esta introdução, bem como informações sobre a engine, sua história, seu licenciamento, autores, etc. Também contém as [Perguntas frequentes](#).
- **Primeiros passos** contém todas as informações necessárias para usar a engine para criar jogos. Ele começa com a seção [Introdução](#), que deve ser o ponto de partida para todos os novos usuários. **Este é o melhor lugar para começar se você é iniciante!**
- O **Manual** pode ser lido ou referenciado conforme necessário, em qualquer ordem. Ele contém tutoriais e documentação para funcionalidades específicas.
- **Detalhes da Engine** contém seções destinadas a usuários avançados e colaboradores, com informações sobre como compilar a engine, trabalhar no editor ou desenvolver módulos de C++.
- **Comunidade** é dedicado à vida da comunidade do Godot e contém uma lista de tutoriais e materiais de terceiros recomendados fora desta documentação. Também fornece detalhes sobre a Biblioteca de Recursos. Anteriormente, listava algumas comunidades online do Godot, que agora estão disponíveis no [site do Godot](#).
- Finalmente, a **Referência de classe** documenta a API do Godot completamente, também disponível diretamente dentro do editor de script da engine. Você pode encontrar informações sobre todas as classes, funções, sinais e afins aqui.

Além desta documentação, você também pode querer dar uma olhada nos vários [Projetos de demonstração do Godot](#).

Sobre esta documentação

Os membros da comunidade do Godot Engine continuamente escrevem, corrigem, editam e aprimoram esta documentação. Estamos sempre buscando mais ajuda. Você também pode contribuir abrindo issues no Github ou traduzindo a documentação para o seu idioma. Se você estiver interessado em ajudar, consulte [How to contribute](#) e [Writing documentation](#), ou entre em contato com a [equipe de documentação](#) no [Chat de Contribuidores do Godot](#).

Todo o conteúdo da documentação está sob a licença permissiva Creative Commons Attribution 3.0 ([CC-BY 3.0](#)), atribuído a "Juan Linietsky, Ariel Manzur e a comunidade do Godot Engine" a menos que outra licença seja especificada.

Divirta-se lendo e fazendo jogos com o Godot Engine!

Interfaces em Godot

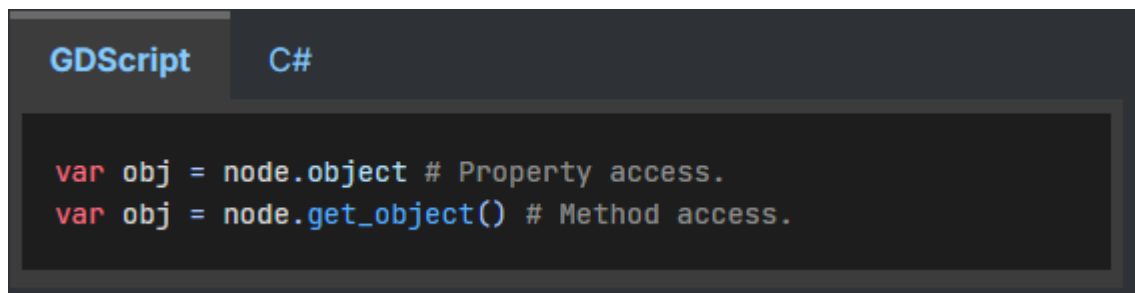
Muitas vezes são necessários scripts que dependem de outros objetos para funcionalidades. Este processo tem 2 partes:

1. Adquirir uma referência ao objeto que presumivelmente possui as funcionalidades.
2. Acessar os dados ou a lógica do objeto.

O resto deste tutorial descreve as várias formas de fazer tudo isto.

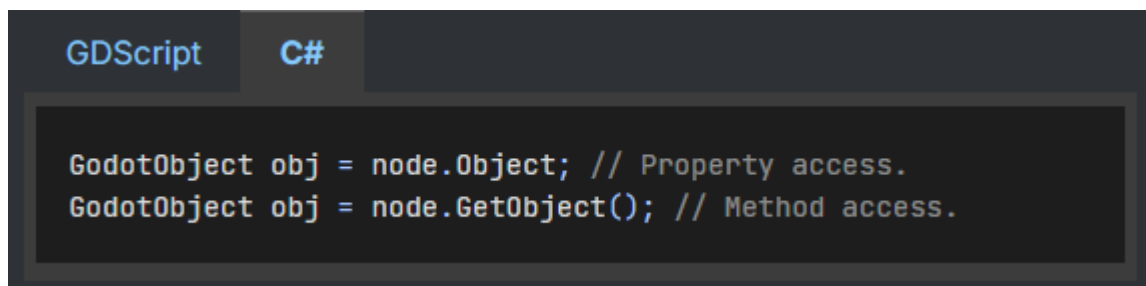
Adquirindo referências de objetos

Para todos os [Objects](#), a forma mais básica de referenciá-los é obtendo uma referência para um objeto existente de outra instância adquirida.



The screenshot shows the Godot code editor with the 'GDScript' tab selected. The code is as follows:

```
var obj = node.object # Property access.  
var obj = node.get_object() # Method access.
```



The screenshot shows the Godot code editor with the 'C#' tab selected. The code is as follows:

```
GodotObject obj = node.Object; // Property access.  
GodotObject obj = node.GetObject(); // Method access.
```

The same principle applies for [RefCounted](#) objects. While users often access [Node](#) and [Resource](#) this way, alternative measures are available.

Em vez de acesso a propriedade ou método, pode-se obter Recursos por acesso de carregamento.

GDScript**C#**

```
# If you need an "export const var" (which doesn't exist), use a conditional
# setter for a tool script that checks if it's executing in the editor.
# The `@tool` annotation must be placed at the top of the script.
@tool

# Load resource during scene load.
var preres = preload(path)
# Load resource when program reaches statement.
var res = load(path)

# Note that users load scenes and scripts, by convention, with PascalCase
# names (like typename), often into constants.
const MyScene = preload("my_scene.tscn") # Static load
const MyScript = preload("my_script.gd")

# This type's value varies, i.e. it is a variable, so it uses snake_case.
@export var script_type: Script

# Must configure from the editor, defaults to null.
@export var const_script: Script:
    set(value):
        if Engine.is_editor_hint():
            const_script = value

# Warn users if the value hasn't been set.
func _get_configuration_warnings():
    if not const_script:
        return ["Must initialize property 'const_script'."]

    return []
```

```
GDScript  C#

// Tool script added for the sake of the "const [Export]" example.
[Tool]
public MyType
{
    // Property initializations load during Script instancing, i.e. .new().
    // No "preload" loads during scene load exists in C#.

    // Initialize with a value. Editable at runtime.
    public Script MyScript = GD.Load<Script>("res://Path/To/MyScript.cs");

    // Initialize with same value. Value cannot be changed.
    public readonly Script MyConstScript = GD.Load<Script>("res://Path/To/MyScript.cs");

    // Like 'readonly' due to inaccessible setter.
    // But, value can be set during constructor, i.e. MyType().
    public Script MyNoSetScript { get; } = GD.Load<Script>("res://Path/To/MyScript.cs");

    // If need a "const [Export]" (which doesn't exist), use a
    // conditional setter for a tool script that checks if it's executing
    // in the editor.
    private PackedScene _enemyScn;

    [Export]
    public PackedScene EnemyScn
    {
        get { return _enemyScn; }
        set
        {
            if (Engine.IsEditorHint())
            {
                _enemyScn = value;
            }
        }
    };

    // Warn users if the value hasn't been set.
    public string[] _GetConfigurationWarnings()
    {
        if (EnemyScn == null)
        {
            return ["Must initialize property 'EnemyScn'."];
        }
        return [];
    }
}
```

Observe o seguinte:

1. Existem muitas maneiras de uma linguagem carregar tais recursos.
2. Ao projetar como os objetos acessarão os dados, não se esqueça de que também se pode passar recursos como referências.
3. Tenha em mente que carregar um recurso busca a instância do recurso em cache mantida pelo motor. Para obter um novo objeto, deve-se [duplicar](#) uma referência existente ou instanciar uma do zero com `new()`.

Nós também têm um ponto de acesso alternativo: a SceneTree.

GDScript

C#

```
extends Node

# Slow.
func dynamic_lookup_with_dynamic_nodepath():
    print(get_node("Child"))

# Faster. GDScript only.
func dynamic_lookup_with_cached_nodepath():
    print($Child)

# Fastest. Doesn't break if node moves later.
# Note that '@onready' annotation is GDScript-only.
# Other languages must do...
#     var child
#     func _ready():
#         child = get_node("Child")
@onready var child = $Child
func lookup_and_cache_for_future_access():
    print(child)

# Fastest. Doesn't break if node is moved in the Scene tree dock.
# Node must be selected in the inspector as it's an exported property.
@export var child: Node
func lookup_and_cache_for_future_access():
    print(child)

# Delegate reference assignment to an external source.
# Con: need to perform a validation check.
# Pro: node makes no requirements of its external structure.
#     'prop' can come from anywhere.
var prop
func call_me_after_prop_is_initialized_by_parent():
    # Validate prop in one of three ways.

    # Fail with no notification.
    if not prop:
        return

    # Fail with an error message.
    if not prop:
        printerr("'prop' wasn't initialized")
        return

    # Fail and terminate.
    # NOTE: Scripts run from a release export template don't run `assert`s.
    assert(prop, "'prop' wasn't initialized")

# Use an autoload.
# Dangerous for typical nodes, but useful for true singleton nodes
# that manage their own data and don't interfere with other objects.
func reference_a_global_autoloaded_variable():
    print(globals)
    print(globals.prop)
    print(globals.my_getter())
```

```
GDScript  C#

using Godot;
using System;
using System.Diagnostics;

public class MyNode : Node
{
    // Slow
    public void DynamicLookupWithDynamicNodePath()
    {
        GD.Print(GetNode("Child"));
    }

    // Fastest. Lookup node and cache for future access.
    // Doesn't break if node moves later.
    private Node _child;
    public void _Ready()
    {
        _child = GetNode("Child");
    }
    public void LookupAndCacheForFutureAccess()
    {
        GD.Print(_child);
    }

    // Delegate reference assignment to an external source.
    // Con: need to perform a validation check.
    // Pro: node makes no requirements of its external structure.
    // 'prop' can come from anywhere.
    public object Prop { get; set; }
    public void CallMeAfterPropIsInitializedByParent()
    {
        // Validate prop in one of three ways.

        // Fail with no notification.
        if (prop == null)
        {
            return;
        }

        // Fail with an error message.
        if (prop == null)
        {
            GD.PrintErr("'Prop' wasn't initialized");
            return;
        }

        // Fail with an exception.
        if (prop == null)
        {
            throw new InvalidOperationException("'Prop' wasn't initialized.");
        }

        // Fail and terminate.
        // Note: Scripts run from a release export template don't run 'Debug.Assert's.
        Debug.Assert(Prop, "'Prop' wasn't initialized");
    }

    // Use an autoload.
    // Dangerous for typical nodes, but useful for true singleton nodes
    // that manage their own data and don't interfere with other objects.
    public void ReferenceAGlobalAutoloadedVariable()
    {
        MyNode gLabels = GetNode<MyNode>("/root/GLabels");
        GD.Print(gLabels);
        GD.Print(gLabels.Prop);
        GD.Print(gLabels.MyGetter());
    }
}

};
```

Acessando dados ou lógica a partir de um objeto

A API de scripting do Godot é duck-typed. Isto significa que se um script executa uma operação, o Godot não valida que ele suporta a operação por **type***. Em vez disso, ele verifica se o objeto **implementa** o método individual.

Por exemplo, a classe [CanvasItem](#) tem uma propriedade `visible`. Todas as propriedades expostas à API de scripting são na verdade um par de setter e getter vinculado a um nome. Se alguém tentasse acessar [CanvasItem.visible](#), então o Godot faria as seguintes verificações, na ordem:

- Se o objeto tiver um script anexado, ele tentará definir a propriedade através do script. Isto deixa aberta a oportunidade para os scripts sobrepor uma propriedade definida em um objeto base sobrepondo o método setter para a propriedade.
- Se o script não tiver a propriedade, ele realiza uma pesquisa HashMap na ClassDB pela propriedade "visible" na classe CanvasItem e todos os seus tipos herdados. Se encontrada, ela chamará o setter ou getter vinculado. Para mais informações sobre HashMaps, veja a documentação [preferências de dados](#).
- Se não for encontrado, ele faz uma verificação explícita para ver se o usuário deseja acessar as propriedades "script" ou "meta".
- Caso contrário, ele verifica se há uma implementação `_set/ _get` (dependendo do tipo de acesso) no CanvasItem e seus tipos herdados. Estes métodos podem executar uma lógica que dá a impressão de que o objeto possui uma propriedade. Este também é o caso com o método `_get_property_list`.
 - Observe que isso acontece mesmo para nomes de símbolos não permitidos, como nomes que começam com um dígito ou contêm uma barra.

Como resultado, este sistema duck-typed pode localizar uma propriedade no script, na classe do objeto ou em qualquer classe que o objeto herda, mas apenas para coisas que estendem de Object.

O Godot fornece uma variedade de opções para realizar verificações durante a execução nestes acessos:

- A duck-typed property access. These will be property checks (as described above). If the operation isn't supported by the object, execution will halt.

GDScript

C#

```
# All Objects have duck-typed get, set, and call wrapper methods.
get_parent().set("visible", false)

# Using a symbol accessor, rather than a string in the method call,
# will implicitly call the `set` method which, in turn, calls the
# setter method bound to the property through the property lookup
# sequence.
get_parent().visible = false

# Note that if one defines a _set and _get that describe a property's
# existence, but the property isn't recognized in any _get_property_list
# method, then the set() and get() methods will work, but the symbol
# access will claim it can't find the property.
```

```
GDScript  C#

// All Objects have duck-typed Get, Set, and Call wrapper methods.
GetParent().Set("visible", false);

// C# is a static language, so it has no dynamic symbol access, e.g.
// `GetParent().Visible = false` won't work.
```

Uma verificação de método. No caso de [CanvasItem.visible](#), pode-se acessar os métodos, `set_visible` e `is_visible` como qualquer outro método.

ODScript

C#

```
var child = get_child(8)

# Dynamic lookup.
child.call("set_visible", false)

# Symbol-based dynamic lookup.
# ODScript aliases this into a 'call' method behind the scenes.
child.set_visible(false)

# Dynamic lookup, checks for method existence first.
if child.has_method("set_visible"):
    child.set_visible(false)

# Cast check, followed by dynamic lookup.
# Useful when you make multiple "safe" calls knowing that the class
# implements them all. No need for repeated checks.
# Tricky if one executes a cast check for a user-defined type as it
# forces more dependencies.
if child is CanvasItem:
    child.set_visible(false)
    child.show_on_top = true

# If one does not wish to fail these checks without notifying users,
# one can use an assert instead. These will trigger runtime errors
# immediately if not true.
assert(child.has_method("set_visible"))
assert(child.is_in_group("offer"))
assert(child is CanvasItem)

# Can also use object labels to imply an interface, i.e. assume it
# implements certain methods.
# There are two types, both of which only exist for Nodes: Names and
# Groups.

# Assuming...
# A "Quest" object exists and 1) that it can "complete" or "fail" and
# that it will have text available before and after each state...

# 1. Use a name.
var quest = $quest
print(quest.text)
quest.complete() # or quest.fail()
print(quest.text) # implied new text content

# 2. Use a group.
for a_child in get_children():
    if a_child.is_in_group("quest"):
        print(quest.text)
        quest.complete() # or quest.fail()
        print(quest.text) # implied new text content

# Note that these interfaces are project-specific conventions the team
# defines (which means documentation! But maybe worth it?).
# Any script that conforms to the documented "interface" of the name or
# a group can fill in for it.
```

```

GOScript  C#

Node child = GetChild(B);

// Dynamic lookup.
child.Call("SetVisible", false);

// Dynamic lookup, checks for method existence first.
if (child.HasMethod("SetVisible"))
{
    child.Call("SetVisible", false);
}

// Use a group as if it were an "interface", i.e. assume it implements
// certain methods.
// Requires good documentation for the project to keep it reliable
// (unless you make editor tools to enforce it at editor time).
// Note, this is generally not as good as using an actual interface in
// C#, but you can't set C# interfaces from the editor since they are
// language-level features.
if (child.IsInGroup("Offer"))
{
    child.Call("Accept");
    child.Call("Reject");
}

// Cast check, followed by static lookup.
CanvasItem ci = GetParent() as CanvasItem;
if (ci != null)
{
    ci.SetVisible(false);

    // useful when you need to make multiple safe calls to the class
    ci.ShowOnTop = true;
}

// If one does not wish to fail these checks without notifying users,
// one can use an assert instead. These will trigger runtime errors
// immediately if not true.
Debug.Assert(child.HasMethod("set_visible"));
Debug.Assert(child.IsInGroup("offer"));
Debug.Assert(CanvasItem.InstanceHas(child));

// Can also use object labels to imply an interface, i.e. assume it
// implements certain methods.
// There are two types, both of which only exist for Nodes: Names and
// Groups.

// Assuming...
// A "Quest" object exists and 1) that it can "Complete" or "Fail" and
// that it will have Text available before and after each state...

// 1. Use a name.
Node quest = GetNode("Quest");
GD.Print(quest.Get("Text"));
quest.Call("Complete"); // or "Fail".
GD.Print(quest.Get("Text")); // Implied new text content.

// 2. Use a group.
foreach (Node AChild in GetChildren())
{
    if (AChild.IsInGroup("quest"))
    {
        GD.Print(quest.Get("Text"));
        quest.Call("Complete"); // or "Fail".
        GD.Print(quest.Get("Text")); // Implied new text content.
    }
}

// Note that these interfaces are project-specific conventions the team
// defines (which means documentation! But maybe worth it?).
// Any script that conforms to the documented "interface" of the
// name or group can fill in for it. Also note that in C#, these methods
// will be slower than static accesses with traditional interfaces.

```

Outsource the access to a [Callable](#). These may be useful in cases where one needs the max level of freedom from dependencies. In this case, one relies on an external context to setup the method.

GDScript

C#

```
# child.gd
extends Node
var fn = null

func my_method():
    if fn:
        fn.call()

# parent.gd
extends Node

@onready var child = $Child

func _ready():
    child.fn = print_me
    child.my_method()

func print_me():
    print(name)
```

```
GDScript  C#

// Child.cs
using Godot;

public partial class Child : Node
{
    public Callable? Callable { get; set; }

    public void MyMethod()
    {
        Callable?.Call();
    }
}

// Parent.cs
using Godot;

public partial class Parent : Node
{
    private Child _child;

    public void _Ready()
    {
        _child = GetNode<Child>("Child");
        _child.Callable = Callable.From(PrintMe);
        _child.MyMethod();
    }

    public void PrintMe()
    {
        GD.Print(Name);
    }
}
```

Estas estratégias contribuem para o design flexível do Godot. Entre elas, os usuários têm uma grande variedade de ferramentas para atender às suas necessidades específicas.

Muitas vezes são necessários scripts que dependem de outros objetos para funcionalidades. Este processo tem 2 partes:

1. Adquirir uma referência ao objeto que presumivelmente possui as funcionalidades.
2. Acessar os dados ou a lógica do objeto.

O resto deste tutorial descreve as várias formas de fazer tudo isto.

Adquirindo referências de objetos

Para todos os [Objects](#), a forma mais básica de referenciá-los é obtendo uma referência para um objeto existente de outra instância adquirida.

GDScript**C#**

```
var obj = node.object # Property access.  
var obj = node.get_object() # Method access.
```

GDScript**C#**

```
GodotObject obj = node.Object; // Property access.  
GodotObject obj = node.GetObject(); // Method access.
```

The same principle applies for [RefCounted](#) objects. While users often access [Node](#) and [Resource](#) this way, alternative measures are available.

Em vez de acesso a propriedade ou método, pode-se obter Recursos por acesso de carregamento.

GDScript**C#**

```
# If you need an "export const var" (which doesn't exist), use a conditional  
# setter for a tool script that checks if it's executing in the editor.  
# The '@tool' annotation must be placed at the top of the script.  
@tool  
  
# Load resource during scene load.  
var preres = preload(path)  
# Load resource when program reaches statement.  
var res = load(path)  
  
# Note that users load scenes and scripts, by convention, with PascalCase  
# names (like typenames), often into constants.  
const MyScene = preload("my_scene.tscn") # Static load  
const MyScript = preload("my_script.gd")  
  
# This type's value varies, i.e. it is a variable, so it uses snake_case.  
@export var script_type: Script  
  
# Must configure from the editor, defaults to null.  
@export var const_script: Script:  
    set(value):  
        if Engine.is_editor_hint():  
            const_script = value  
  
# Warn users if the value hasn't been set.  
func _get_configuration_warnings():  
    if not const_script:  
        return ["Must initialize property 'const_script'."]   
  
    return []
```

GDScript

C#

```
// Tool script added for the sake of the "const [Export]" example.
[Tool]
public MyType
{
    // Property initializations load during Script instancing, i.e. .new().
    // No "preload" loads during scene load exists in C#.

    // Initialize with a value. Editable at runtime.
    public Script MyScript = GD.Load<Script>("res://Path/To/MyScript.cs");

    // Initialize with same value. Value cannot be changed.
    public readonly Script MyConstScript = GD.Load<Script>("res://Path/To/MyScript.cs");

    // Like 'readonly' due to inaccessible setter.
    // But, value can be set during constructor, i.e. MyType().
    public Script MyNoSetScript { get; } = GD.Load<Script>("res://Path/To/MyScript.cs");

    // If need a "const [Export]" (which doesn't exist), use a
    // conditional setter for a tool script that checks if it's executing
    // in the editor.
    private PackedScene _enemyScn;

    [Export]
    public PackedScene EnemyScn
    {
        get { return _enemyScn; }
        set
        {
            if (Engine.IsEditorHint())
            {
                _enemyScn = value;
            }
        }
    };

    // Warn users if the value hasn't been set.
    public string[] _GetConfigurationWarnings()
    {
        if (EnemyScn == null)
        {
            return ["Must initialize property 'EnemyScn'."];
        }
        return [];
    }
}
```

Observe o seguinte:

1. Existem muitas maneiras de uma linguagem carregar tais recursos.
2. Ao projetar como os objetos acessarão os dados, não se esqueça de que também se pode passar recursos como referências.
3. Tenha em mente que carregar um recurso busca a instância do recurso em cache mantida pelo motor. Para obter um novo objeto, deve-se [duplicar](#) uma referência existente ou instanciar uma do zero com `new()`.

Nós também têm um ponto de acesso alternativo: a SceneTree.

GDScript

C#

```
extends Node

# Slow.
func dynamic_lookup_with_dynamic_nodepath():
    print(get_node("Child"))

# Faster. GDScript only.
func dynamic_lookup_with_cached_nodepath():
    print($Child)

# Fastest. Doesn't break if node moves later.
# Note that '@onready' annotation is GDScript-only.
# Other languages must do...
#     var child
#     func _ready():
#         child = get_node("Child")
@onready var child = $Child
func lookup_and_cache_for_future_access():
    print(child)

# Fastest. Doesn't break if node is moved in the Scene tree dock.
# Node must be selected in the inspector as it's an exported property.
@export var child: Node
func lookup_and_cache_for_future_access():
    print(child)

# Delegate reference assignment to an external source.
# Con: need to perform a validation check.
# Pro: node makes no requirements of its external structure.
#     'prop' can come from anywhere.
var prop
func call_me_after_prop_is_initialized_by_parent():
    # Validate prop in one of three ways.

    # Fail with no notification.
    if not prop:
        return

    # Fail with an error message.
    if not prop:
        printerr("'prop' wasn't initialized")
        return

    # Fail and terminate.
    # NOTE: Scripts run from a release export template don't run `assert`s.
    assert(prop, "'prop' wasn't initialized")

# Use an autoload.
# Dangerous for typical nodes, but useful for true singleton nodes
# that manage their own data and don't interfere with other objects.
func reference_a_global_autoloaded_variable():
    print(globals)
    print(globals.prop)
    print(globals.my_getter())
```

```
GDScript C#

using Godot;
using System;
using System.Diagnostics;

public class MyNode : Node
{
    // Slow
    public void DynamicLookupWithDynamicNodePath()
    {
        GD.Print(SetNode("Child"));
    }

    // Fastest. Lookup node and cache for future access.
    // Doesn't break if node moves later.
    private Node _child;
    public void _Ready()
    {
        _child = SetNode("Child");
    }
    public void LookupAndCacheForFutureAccess()
    {
        GD.Print(_child);
    }

    // Delegate reference assignment to an external source.
    // Con: need to perform a validation check.
    // Pro: node makes no requirements of its external structure.
    // 'prop' can come from anywhere.
    public object Prop { get; set; }
    public void CallMeAfterPropIsInitializedByParent()
    {
        // Validate prop in one of three ways.

        // Fail with no notification.
        if (prop == null)
        {
            return;
        }

        // Fail with an error message.
        if (prop == null)
        {
            GD.PrintErr("'Prop' wasn't initialized");
            return;
        }

        // Fail with an exception.
        if (prop == null)
        {
            throw new InvalidOperationException("'Prop' wasn't initialized.");
        }

        // Fail and terminate.
        // Note: Scripts run from a release export template don't run Debug.Assert's.
        Debug.Assert(Prop, "'Prop' wasn't initialized");
    }

    // Use an autoload.
    // Dangerous for typical nodes, but useful for true singleton nodes
    // that manage their own data and don't interfere with other objects.
    public void ReferenceAGlobalAutoloadedVariable()
    {
        MyNode globals = SetNode<MyNode>("/root/Globals");
        GD.Print(globals);
        GD.Print(globals.Prop);
        GD.Print(globals.MyGetter());
    }
};
```

Acessando dados ou lógica a partir de um objeto

A API de scripting do Godot é duck-typed. Isto significa que se um script executa uma operação, o Godot não valida que ele suporta a operação por **type***. Em vez disso, ele verifica se o objeto **implementa** o método individual.

Por exemplo, a classe [CanvasItem](#) tem uma propriedade `visible`. Todas as propriedades expostas à API de scripting são na verdade um par de setter e getter vinculado a um nome. Se alguém tentasse acessar [CanvasItem.visible](#), então o Godot faria as seguintes verificações, na ordem:

- Se o objeto tiver um script anexado, ele tentará definir a propriedade através do script. Isto deixa aberta a oportunidade para os scripts sobrepor uma propriedade definida em um objeto base sobrepondo o método setter para a propriedade.
- Se o script não tiver a propriedade, ele realiza uma pesquisa HashMap na ClassDB pela propriedade "visible" na classe CanvasItem e todos os seus tipos herdados. Se encontrada, ela chamará o setter ou getter vinculado. Para mais informações sobre HashMaps, veja a documentação [preferências de dados](#).
- Se não for encontrado, ele faz uma verificação explícita para ver se o usuário deseja acessar as propriedades "script" ou "meta".
- Caso contrário, ele verifica se há uma implementação `_set/ _get` (dependendo do tipo de acesso) no CanvasItem e seus tipos herdados. Estes métodos podem executar uma lógica que dá a impressão de que o objeto possui uma propriedade. Este também é o caso com o método `_get_property_list`.
 - Observe que isso acontece mesmo para nomes de símbolos não permitidos, como nomes que começam com um dígito ou contêm uma barra.

Como resultado, este sistema duck-typed pode localizar uma propriedade no script, na classe do objeto ou em qualquer classe que o objeto herda, mas apenas para coisas que estendem de Object.

O Godot fornece uma variedade de opções para realizar verificações durante a execução nestes acessos:

- A duck-typed property access. These will be property checks (as described above). If the operation isn't supported by the object, execution will halt.

GDScript**C#**

```
# All Objects have duck-typed get, set, and call wrapper methods.
get_parent().set("visible", false)

# Using a symbol accessor, rather than a string in the method call,
# will implicitly call the `set` method which, in turn, calls the
# setter method bound to the property through the property lookup
# sequence.
get_parent().visible = false

# Note that if one defines a _set and _get that describe a property's
# existence, but the property isn't recognized in any _get_property_list
# method, then the set() and get() methods will work, but the symbol
# access will claim it can't find the property.
```

GDScript**C#**

```
// All Objects have duck-typed Get, Set, and Call wrapper methods.
GetParent().Set("visible", false);

// C# is a static language, so it has no dynamic symbol access, e.g.
// `GetParent().Visible = false` won't work.
```

Uma verificação de método. No caso de [CanvasItem.visible](#), pode-se acessar os métodos, `set_visible` e `is_visible` como qualquer outro método.

GDScript

C#

```
var child = get_child(0)

# Dynamic lookup.
child.call("set_visible", false)

# Symbol-based dynamic lookup.
# GDScript aliases this into a 'call' method behind the scenes.
child.set_visible(false)

# Dynamic lookup, checks for method existence first.
if child.has_method("set_visible"):
    child.set_visible(false)

# Cast check, followed by dynamic lookup.
# Useful when you make multiple "safe" calls knowing that the class
# implements them all. No need for repeated checks.
# Tricky if one executes a cast check for a user-defined type as it
# forces more dependencies.
if child is CanvasItem:
    child.set_visible(false)
    child.show_on_top = true

# If one does not wish to fail these checks without notifying users,
# one can use an assert instead. These will trigger runtime errors
# immediately if not true.
assert(child.has_method("set_visible"))
assert(child.is_in_group("offer"))
assert(child is CanvasItem)

# Can also use object labels to imply an interface, i.e. assume it
# implements certain methods.
# There are two types, both of which only exist for Nodes: Names and
# Groups.

# Assuming...
# A "Quest" object exists and 1) that it can "complete" or "fail" and
# that it will have text available before and after each state...

# 1. Use a name.
var quest = $Quest
print(quest.text)
quest.complete() # or quest.fail()
print(quest.text) # implied new text content

# 2. Use a group.
for a_child in get_children():
    if a_child.is_in_group("quest"):
        print(quest.text)
        quest.complete() # or quest.fail()
        print(quest.text) # implied new text content

# Note that these interfaces are project-specific conventions the team
# defines (which means documentation! But maybe worth it?).
# Any script that conforms to the documented "interface" of the name or
# group can fill in for it.
```

GDScript

C#

```
Node child = GetChild(0);

// Dynamic lookup.
child.Call("SetVisible", false);

// Dynamic lookup, checks for method existence first.
if (child.HasMethod("SetVisible"))
{
    child.Call("SetVisible", false);
}

// Use a group as if it were an "interface", i.e. assume it implements
// certain methods.
// Requires good documentation for the project to keep it reliable
// (unless you make editor tools to enforce it at editor time).
// Note, this is generally not as good as using an actual interface in
// C#, but you can't set C# interfaces from the editor since they are
// language-level features.
if (child.IsInGroup("Offer"))
{
    child.Call("Accept");
    child.Call("Reject");
}

// Cast check, followed by static lookup.
CanvasItem ci = GetParent() as CanvasItem;
if (ci != null)
{
    ci.SetVisible(false);

    // useful when you need to make multiple safe calls to the class
    ci.ShowOnTop = true;
}

// If one does not wish to fail these checks without notifying users,
// one can use an assert instead. These will trigger runtime errors
// immediately if not true.
Debug.Assert(child.HasMethod("set_visible"));
Debug.Assert(child.IsInGroup("offer"));
Debug.Assert(CanvasItem.InstanceHas(child));

// Can also use object labels to imply an interface, i.e. assume it
// implements certain methods.
// There are two types, both of which only exist for Nodes: Names and
// Groups.

// Assuming...
// A "Quest" object exists and 1) that it can "Complete" or "Fail" and
// that it will have Text available before and after each state...

// 1. Use a name.
Node quest = GetNode("Quest");
GD.Print(quest.Get("Text"));
quest.Call("Complete"); // or "Fail".
GD.Print(quest.Get("Text")); // Implied new text content.

// 2. Use a group.
foreach (Node AChild in GetChildren())
{
    if (AChild.IsInGroup("quest"))
    {
        GD.Print(quest.Get("Text"));
        quest.Call("Complete"); // or "Fail".
        GD.Print(quest.Get("Text")); // Implied new text content.
    }
}

// Note that these interfaces are project-specific conventions the team
// defines (which means documentation! But maybe worth it?).
// Any script that conforms to the documented "interface" of the
// name or group can fill in for it. Also note that in C#, these methods
// will be slower than static accesses with traditional interfaces.
```

Outsource the access to a [Callable](#). These may be useful in cases where one needs the max level of freedom from dependencies. In this case, one relies on an external context to setup the method.

GDScript**C#**

```
# child.gd
extends Node
var fn = null

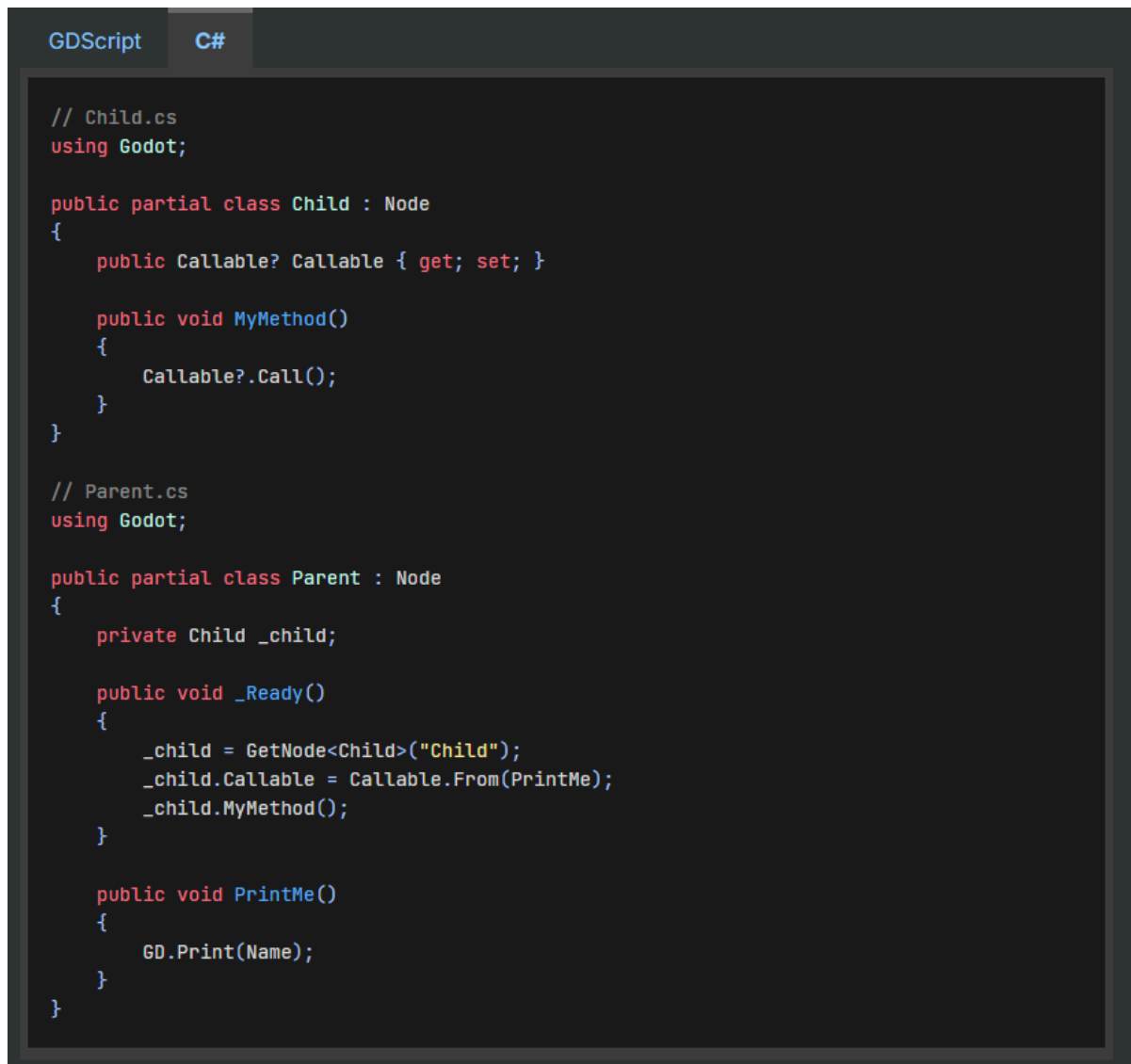
func my_method():
    if fn:
        fn.call()

# parent.gd
extends Node

@onready var child = $Child

func _ready():
    child.fn = print_me
    child.my_method()

func print_me():
    print(name)
```



```
// Child.cs
using Godot;

public partial class Child : Node
{
    public Callable? Callable { get; set; }

    public void MyMethod()
    {
        Callable?.Call();
    }
}

// Parent.cs
using Godot;

public partial class Parent : Node
{
    private Child _child;

    public void _Ready()
    {
        _child = GetNode<Child>("Child");
        _child.Callable = Callable.From(PrintMe);
        _child.MyMethod();
    }

    public void PrintMe()
    {
        GD.Print(Name);
    }
}
```

Estas estratégias contribuem para o design flexível do Godot. Entre elas, os usuários têm uma grande variedade de ferramentas para atender às suas necessidades específicas.